

BankBook

Grado Superior de Desarrollo de Aplicaciones Web

Aplicación Web para la Gestión del Préstamo de Libros

BancoLibros – Versión Web (PHP / HTML / JavaScript / MySQL)

Curso académico 2024 – 2025

IES Arcipreste de Hita

**Autores: Francisco Pardo Doñoro e
Ivan García López**

Tutor: Aaron Montalvo

Índice de Contenido

Índice de Contenido	2
1. INTRODUCCIÓN	4
1.1 Contexto y Origen del Proyecto	4
1.2 Análisis de Posibilidades	4
1.2.1 Uso de una Hoja de Cálculo (Excel) o Gestión Manual	4
1.2.2 Uso de Access o Base de Datos Local	5
1.2.3 Creación de una Aplicación Web Propia	5
1.3 Objetivo General del Proyecto dentro del ámbito	5
2. Definición del Proyecto	6
2.1 Alcance y Límites	6
2.2 Catálogo de Objetivos	6
2.2.1 Meta Principal	6
2.2.2 Hitos Técnicos y Operativos	7
3. INGENIERÍA DE REQUISITOS	7
3.1 Análisis Funcional	7
3.1.1 Especificación de Requisitos del Sistema (ERS)	7
3.2 Análisis Funcional	8
3.3 Estudio de Viabilidad	9
3.3.1 Viabilidades Críticas	9
3.3.2. Marco Legal y Protección de Datos	9
4. ARQUITECTURA Y DISEÑO TÉCNICO	10
4.1. Ecosistema Tecnológico	10
4.1.1. Arquitectura del Sistema (Patrón MVC)	10
4.1.2. Stack de Desarrollo y Lenguajes	11
4.1.3. Herramientas de Ingeniería	11
4.2. Entorno de Explotación e Implantación	12
4.3. Diseño de Datos y UX	12
4.3.1. Diagrama Entidad-Relación (E/R)	13
4.3.2. Diagrama de Casos de Uso	13
4.3.3. Diagrama de Navegación	14
5. DESARROLLO Y LÓGICA DE APLICACIÓN	15
5.1 Estructura de la Base de Datos	15
5.2 Capa de Acceso a Datos (PDO)	17
5.3 Desarrollo del Backend y Patrón MVC	18
5.3.1. Estructura de Directorios	18
5.3.2. Implementación de Modelos	19
5.3.3. Implementación de Controladores	21
5.3.4. Gestión de Vistas y Renderizado (frontend sin acabar)	22
5.4 Sistema de Enrutamiento y Definición de Rutas	22
5.4.1. Estructura de Rutas por Niveles de Acceso	22
5.4.2. Ventajas del Enrutamiento Dinámico	22
5.4 Seguridad y Control de Acceso	23

5.5.1. Autenticación y Gestión de Sesiones	23
5.5.2. Protección contra Vulnerabilidades	23
5.5.3. Protección contra Vulnerabilidades	23
6. Pruebas del Proyecto	24
6.1 Estrategia de Pruebas	24
6.2 Batería de Pruebas Funcionales	24
6.3 Pruebas de Seguridad	25
6.4 Pruebas de Compatibilidad	26
6.5 Incidencias Detectadas y Resolución	26
6.6 Indicadores de Calidad	27
7. Implantación del Proyecto	28
7.1 Plan de Implantación	28
7.2 Manual de Instalación	28
7.2.1 Requisitos Previos	28
7.2.2 Pasos de Instalación	28
7.2.3 Configuración del Fichero Principal	29
7.3 Mejoras Posibles	29
8. Manual de Usuario	30
8.1 Acceso al Sistema	30
8.2 Panel Principal (Dashboard)	30
8.3 Gestión de Alumnos	30
8.4 Gestión de Libros	30
8.5 Gestión de Préstamos	30
8.6 Listados e Informes	31
8.7 Administración del Sistema	31
9. Conclusiones	32
9.1 Síntesis de Resultados	32
9.2 Valoración Personal	32
9.3 Líneas Futuras	32
10. Webgrafía	33
11. Anexos	34
Anexo I: Formulario de Contrato de Cesión de Libros	34
Anexo II: Estructura del Fichero CSV de Importación de Alumnos (Delphos)	34
Anexo III: Ejemplo de Formulario de Satisfacción	35

1. INTRODUCCIÓN

1.1 Contexto y Origen del Proyecto

El proyecto BancoLibros surge como una respuesta técnica a la necesidad de transformación digital dentro de la operativa administrativa de un centro de Educación Secundaria Obligatoria.

Actualmente, la gestión del programa de préstamo de libros de texto se sustenta en procesos manuales y el uso de hojas de cálculo aisladas; un modelo que presenta carencias críticas en cuanto a la integridad de los datos, la rapidez de consulta y la capacidad de auditoría.

La complejidad del ecosistema escolar exige una herramienta que centralice la información y automatice los flujos de trabajo. El ciclo de vida que cubre esta aplicación abarca desde el principio de curso con la venta y el préstamo de los libros hasta el final del curso con su venta.

Desde el punto de vista tecnológico, la solución se ha edificado sobre un stack robusto y estandarizado, garantizando su viabilidad en entornos de producción reales. En el Backend, se emplea PHP 8.2, aprovechando sus mejoras en rendimiento y tipado, junto a una base de datos relacional MySQL que asegura la persistencia y coherencia de la información. El Frontend se ha diseñado priorizando la experiencia de usuario mediante HTML5, CSS3 con el framework Bootstrap 5 para asegurar una interfaz adaptativa, y JavaScript para la gestión de eventos asíncronos y dinamismo en la interfaz.

1.2 Análisis de Posibilidades

Antes de iniciar el desarrollo de BancoLibros, se realizó un estudio comparativo de las posibles soluciones para corregir las deficiencias del sistema actual. El objetivo era encontrar una herramienta que equilibrara la accesibilidad, la integridad de los datos y la facilidad de mantenimiento.

1.2.1 Uso de una Hoja de Cálculo (Excel) o Gestión Manual

Tras el análisis, se determinó su inviabilidad como solución a largo plazo debido a:

- **Ausencia de Relacionalidad:** Es extremadamente complejo vincular de forma segura alumnos con libros y becas sin duplicar datos o cometer errores de transcripción.
- **Limitación Documental:** La imposibilidad de generar automáticamente contratos de cesión PDF personalizados para cada alumno.
- **Inexistencia de Concurrencia:** Los archivos suelen bloquearse cuando más de un administrativo intenta realizar préstamos simultáneamente.

1.2.2 Uso de Access o Base de Datos Local

Se evaluó el desarrollo de una solución mediante gestores de escritorio como Microsoft Access o SQLite con interfaz local. Aunque mejora la estructura de los datos, se descartó por los siguientes motivos:

- **Dependencia del Hardware:** Obliga a realizar la gestión exclusivamente desde el terminal donde esté instalado el software, limitando la movilidad en el centro.
- **Dificultad de Actualización:** Cualquier mejora o corrección de errores requiere una intervención manual en cada equipo.
- **Curva de Aprendizaje:** Estas interfaces suelen ser menos intuitivas para usuarios no técnicos comparadas con las interfaces web modernas.

1.2.3 Creación de una Aplicación Web Propia

Esta alternativa fue seleccionada como la más eficiente para el entorno escolar. Sus ventajas competitivas son determinantes para el éxito del proyecto:

- **Ubicuidad y Multiplataforma:** Solo requiere un navegador web. El personal de secretaría puede trabajar desde cualquier equipo del instituto sin instalaciones previas.
- **Arquitectura Centralizada:** Tanto la lógica de negocio como la base de datos residen en un servidor (local o remoto), facilitando las copias de seguridad y la integridad de la información.
- **Ecosistema Tecnológico Robusto:** El uso de PHP 8.2 y MySQL garantiza una solución ligera, de código abierto y con una gran facilidad de despliegue en hostings comerciales bajo cPanel.
- **Automatización de Salidas:** Permite la integración de librerías para la generación dinámica de listados e informes de stock y contratos en formatos listos para imprimir.

1.3 Objetivo General del Proyecto dentro del ámbito

El proyecto BancoLibros consiste en el desarrollo de una aplicación web diseñada para su despliegue y explotación en la red interna del centro educativo. Su propósito fundamental es ofrecer una solución funcional, segura y de uso intuitivo que permita gestionar de forma integral el programa de préstamo de libros de texto, centralizando el inventario bibliográfico, el control de beneficiarios por tramos de beca y la administración de las asignaciones.

El ámbito funcional del sistema cubre todo el ciclo operativo, desde la carga inicial de datos y el mantenimiento del catálogo hasta la generación automatizada de informes y documentación administrativa.

La arquitectura de la aplicación diferencia claramente un panel de gestión privada, destinado al personal de secretaría para optimizar los procesos y reducir errores manuales, de un módulo de consulta de stock, garantizando así una administración eficiente y trazable de los recursos del instituto.

2. Definición del Proyecto

2.1 Alcance y Límites

El alcance de BancoLibros comprende la digitalización integral del proceso de préstamo de libros de texto para alumnos de la ESO. El sistema está diseñado para cubrir:

- Gestión de Inventario: Registro y control de stock de libros por curso y materia.
- Gestión de Beneficiarios: Clasificación del alumnado según el tramo de beca asignado por la administración.
- Ciclo de Préstamos: Registro de asignaciones, devoluciones y seguimiento del estado físico de los ejemplares.
- Automatización Documental: Generación de contratos de cesión y listados de control en formato PDF.

Límites: El sistema no contempla la gestión de otros recursos del centro (como material de laboratorio o portátiles) ni la integración directa con la plataforma de notas del instituto, funcionando como una herramienta independiente y especializada para la secretaría.

2.2 Catálogo de Objetivos

2.2.1 Meta Principal

Desarrollar una aplicación web funcional, segura y de fácil uso que permita gestionar de forma integral el programa de préstamo de libros de texto del instituto. El fin último es mejorar la eficiencia del proceso administrativo, reducir los errores manuales y garantizar la trazabilidad de los materiales desde la carga inicial de datos hasta la obtención de documentación final.

2.2.2 Hitos Técnicos y Operativos

Para cumplir con la meta principal, se establecen los siguientes objetivos medibles:

- **Administración de Activos:** Permitir la carga y gestión masiva de datos de materias, cursos y libros al inicio de cada ciclo escolar.
- **Control de Alumnado:** Gestionar el alta, modificación y consulta de datos del alumnado becado de forma ágil.
- **Trazabilidad de Préstamos:** Registrar las asignaciones individuales y controlar las devoluciones con estados específicos (deteriorado, perdido, correcto).
- **Digitalización de Documentos:** Generar e imprimir contratos de cesión de libros listos para la firma del tutor legal.
- **Inteligencia de Datos:** Ofrecer un sistema de búsquedas avanzadas y listados filtrados por múltiples criterios para la toma de decisiones.
- **Seguridad y Persistencia:** Garantizar el acceso mediante autenticación de usuarios y proveer un sistema de copias de seguridad de la base de datos.

3. INGENIERÍA DE REQUISITOS

3.1 Análisis Funcional

El análisis funcional define las capacidades operativas del sistema BancoLibros. Se han identificado las necesidades del personal de secretaría y se han traducido en requisitos que el software debe satisfacer.

3.1.1 Especificación de Requisitos del Sistema (ERS)

Código	Requisito Funcionales	Descripción Detallada
RF-01	Gestión de Catálogo Inicial	Carga y configuración de materias, cursos y libros, definiendo el stock inicial de cada ejemplar.
RF-02	Importación masiva	Capacidad de cargar datos del alumnado mediante el procesamiento de ficheros CSV exportados de Delphos.
RF-03	Gestión de Alumnado	Alta, baja y edición manual de perfiles de alumnos, incluyendo su tramo de beca (I o II).

RF-04	Control de Préstamos	Registro de la asignación de libros a alumnos y gestión del flujo de devoluciones.
RF-05	Estado del Inventario	Monitorización individual del estado de cada libro (entregado, pendiente, deteriorado o perdido).
RF-06	Generación de Contratos	Creación automatizada de documentos de cesión en PDF con los datos del alumno y libros prestados.
RF-07	Búsqueda Avanzada	Sistema de filtrado complejo por alumno, curso, materia y estado para consultas rápidas.
RF-08	Reporting y Listados	Generación de informes exportables en PDF para el control de inventario y préstamos activos.
RF-09	Gestión de Backups	Herramienta para realizar volcados SQL completos de la base de datos para asegurar la integridad.
RF-10	Cierre de Ciclo / Exportación	Preparación y exportación de datos críticos para su uso en la planificación del curso siguiente.

3.2 Análisis Funcional

Estos requisitos definen las restricciones y propiedades del sistema, asegurando que sea robusto y cumpla con la normativa:

- **Seguridad y Autenticación:** El acceso a la gestión está restringido mediante credenciales de usuario y contraseñas cifradas.
- **Cumplimiento Legal (RGPD):** Al tratar datos de menores, el sistema garantiza la privacidad mediante protocolos HTTPS y logs de acceso según la LOPDGDD.
- **Usabilidad:** Interfaz intuitiva basada en Bootstrap 5 que permite operar el sistema con una formación mínima de 2-3 horas.
- **Disponibilidad:** El sistema debe ser accesible de forma concurrente por varios puestos de secretaría sin pérdida de rendimiento.

3.3 Estudio de Viabilidad

3.3.1 Viabilidades Críticas

Viabilidad Técnica: El proyecto emplea un stack estándar (PHP/MySQL) altamente documentado. El uso de herramientas gratuitas o con licencia educativa (PhpStorm, XAMPP) garantiza su desarrollo sin barreras tecnológicas.

Viabilidad Económica: El coste de desarrollo es nulo al ser un proyecto académico. Los gastos de mantenimiento (hosting y dominio) son mínimos (~70€/año), compensados por el gran ahorro de tiempo administrativo.

Viabilidad Operativa: La transición del sistema manual al digital es sencilla. La navegación guiada y los mensajes de confirmación aseguran una alta aceptación por parte del personal.

3.3.2. Marco Legal y Protección de Datos

Dado que BancoLibros gestiona información sensible de menores de edad, el diseño del software integra medidas de seguridad de nivel básico. Se establece la obligatoriedad de certificados SSL/TLS y se prevé la firma de cláusulas informativas para cumplir con el Reglamento General de Protección de Datos (RGPD).

4. ARQUITECTURA Y DISEÑO TÉCNICO

4.1. Ecosistema Tecnológico

Para garantizar la robustez y facilidad de despliegue de **BancoLibros**, se ha seleccionado un stack tecnológico basado en estándares abiertos, estructurado bajo un patrón de diseño profesional que asegura la escalabilidad del sistema.

4.1.1. Arquitectura del Sistema (Patrón MVC)

La aplicación se ha edificado sobre el patrón **Modelo-Vista-Controlador (MVC)**, implementado a través del framework **Laravel**. Esta arquitectura garantiza una separación estricta de responsabilidades, facilitando la escalabilidad del sistema y asegurando que la lógica de negocio, la gestión de datos y la interfaz de usuario funcionen de manera independiente pero coordinada:

- **Modelo (Capa de Datos):** Gestiona la lógica de persistencia en MySQL mediante el ORM **Eloquent**. A diferencia del acceso tradicional, los modelos (como **Libro** o **Alumno**) actúan como objetos inteligentes que no solo representan tablas, sino que definen relaciones complejas y atributos calculados (como la disponibilidad de stock o el estado de vencimiento de los préstamos).
- **Vista (Capa de Presentación):** Implementada mediante el motor de plantillas **Blade**. Esto permite crear interfaces dinámicas en HTML5 y CSS3 (Bootstrap 5) de forma limpia, utilizando herencia de plantillas y componentes reutilizables, lo que asegura una estética uniforme en toda la gestión de la secretaría.
- **Controlador (Capa de Lógica):** Los controladores (ej. **PrestamoController**) actúan como el cerebro del sistema. Orquestan el flujo de trabajo recibiendo las peticiones a través del sistema de enrutamiento de Laravel, validando los datos de entrada mediante *Form Requests* y decidiendo qué respuesta enviar al usuario.

El flujo operativo se basa en peticiones HTTP procesadas por un **Enrutador Central**. Además, para optimizar la experiencia de usuario (UX) en tareas repetitivas de secretaría, el sistema utiliza **peticiones asíncronas (AJAX)**. Estos controladores devuelven respuestas en formato **JSON**, que son procesadas dinámicamente mediante **jQuery/JavaScript** para actualizar la información en pantalla (como los resultados de búsqueda de alumnos) sin necesidad de recargar la página, agilizando así la operativa diaria.

4.1.2. Stack de Desarrollo y Lenguajes

El ecosistema tecnológico de **BancoLibros** se ha seleccionado para garantizar un entorno de desarrollo profesional, seguro y eficiente. La aplicación se basa en un stack **WAMP/LAMP** moderno, utilizando el framework **Laravel** como núcleo integrador:

- **Entorno de Servidor:**
 - **XAMPP 8.2:** Proporciona la infraestructura necesaria con Apache 2.4 como servidor web y **PHP 8.2** como motor de ejecución.
 - **MySQL 8.0:** Motor de base de datos relacional para la persistencia de datos mediante el motor InnoDB.

- **Lenguajes y Tecnologías Core:**
 - **PHP 8.2 (Backend):** Uso de tipado estricto, funciones flecha y las últimas mejoras de rendimiento del lenguaje bajo el framework Laravel.
 - **HTML5 y CSS3 (Frontend):** Estructuración semántica y estilos avanzados.
 - **JavaScript (ES6+):** Lógica en el lado del cliente para interactividad y validaciones dinámicas.

- **Frameworks y Librerías Críticas:**
 - **Laravel 10/11:** Framework principal que aporta el ORM Eloquent, el motor de plantillas Blade y el sistema de seguridad.
 - **Bootstrap 5.3:** Framework de CSS para asegurar una interfaz *responsive* y el uso de componentes UI predefinidos (modales, alertas, tablas).
 - **jQuery 3.7:** Librería utilizada para la manipulación ágil del DOM y la gestión de peticiones asíncronas (AJAX) en el buscador de alumnos.
 - **DomPDF:** Integrado mediante un *wrapper* de Laravel para la generación automatizada de contratos de préstamo y listados de entrega en PDF.
 - **Chart.js:** Utilizado para la representación visual de estadísticas de stock y estado de los libros por curso.
 -

4.1.3. Herramientas de Ingeniería

IDE: VSCODE, utilizado para la codificación, depuración y control de calidad del código.

Modelado y Gestión: phpMyAdmin para la administración de la base de datos y **draw.io** para el diseño de los diagramas relacionales.

Control de Versiones: Git con repositorio remoto en **GitHub**, garantizando la trazabilidad y seguridad del código fuente.

4.2. Entorno de Explotación e Implantación

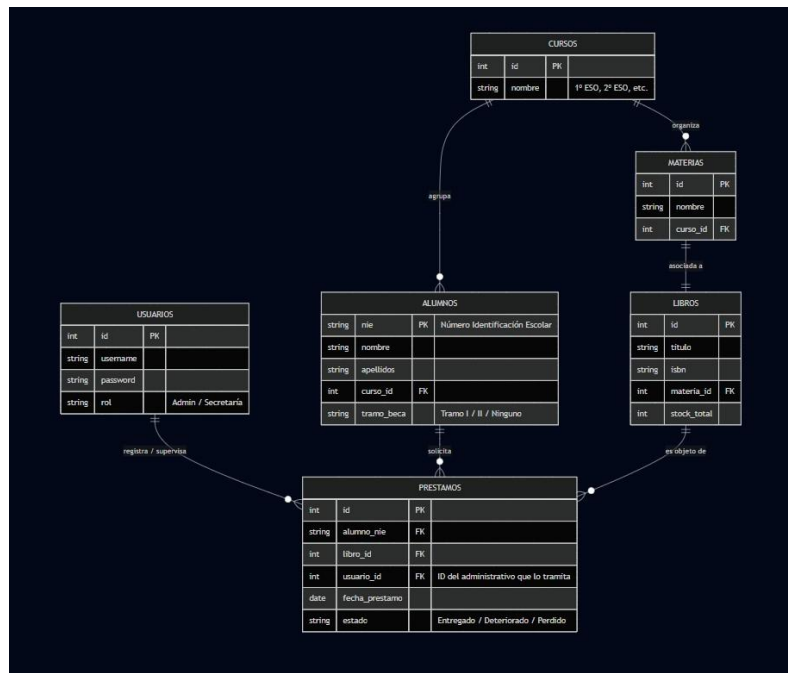
La aplicación está preparada para ser desplegada en servidores de producción reales, cumpliendo con los siguientes requisitos técnicos:

- **Infraestructura:** Servidor web con soporte para PHP 8.2 y extensiones críticas activas (PDO, mbstring, openssl).
- **Base de Datos:** MySQL 8.0 o MariaDB 10.6+.
- **Seguridad:** Es imprescindible la instalación de un certificado SSL/TLS (vía Let's Encrypt) para asegurar el cifrado de datos mediante HTTPS.
- **Administración:** Gestión mediante panel cPanel y acceso vía FTP para la transferencia de archivos.

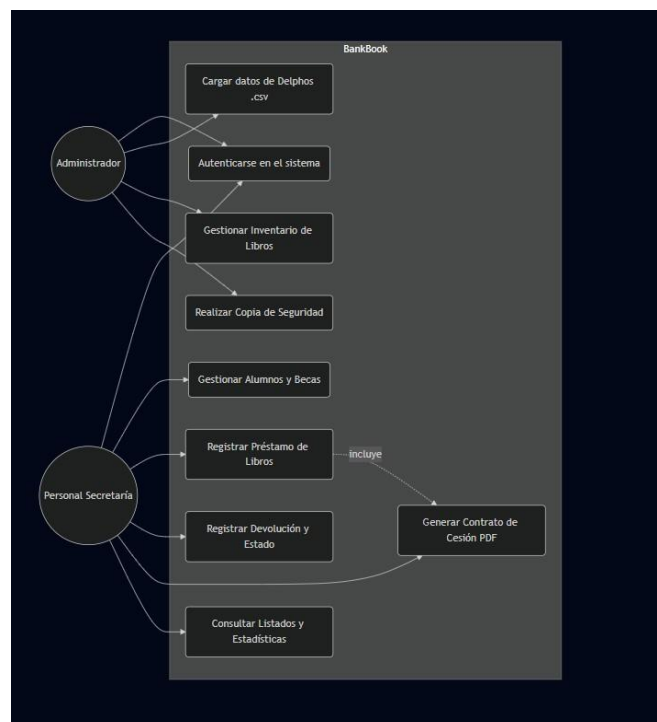
4.3. Diseño de Datos y UX

- **Modelado E/R:** El sistema se basa en una estructura relacional que garantiza la integridad referencial entre las entidades de alumnos, libros y préstamos, asegurando que no existan huérfanos de datos.
- **Interfaz de Usuario (UI/UX):** Se ha diseñado una interfaz limpia y funcional utilizando Bootstrap 5. Se prioriza la velocidad de operación mediante una disposición de elementos intuitiva (Dashboard), feedback visual mediante alertas de estado y un diseño totalmente *responsive* adaptado a diferentes dispositivos.

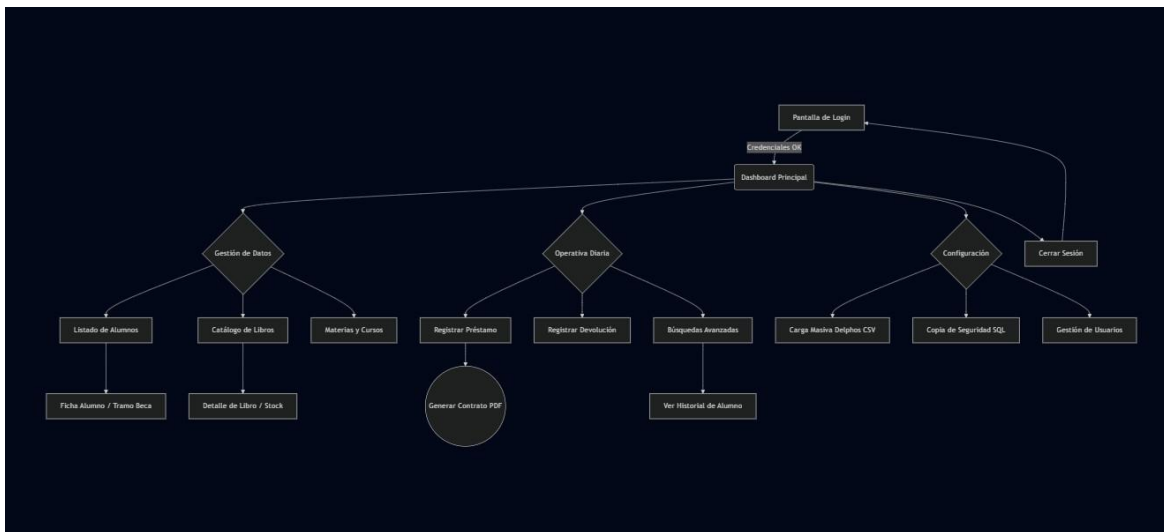
4.3.1. Diagrama Entidad-Relación (E/R)



4.3.2. Diagrama de Casos de Uso



4.3.3. Diagrama de Navegación



5.DESARROLLO Y LÓGICA DE APLICACIÓN

5.1 Estructura de la Base de Datos

Tabla Usuarios:

Campo	Tipo	Nulo	Clave
id	INT	NO	PK
username	VARCHAR (50)	NO	UNI
password	VARCHAR (255)	NO	-
rol	ENUM	NO	-

Tabla Alumnos:

Campo	Tipo	Nulo	Clave
nie	VARCHAR (20)	NO	PK
nombre	VARCHAR (100)	NO	-
apellidos	VARCHAR (100)	NO	-
curso_id	INT	NO	FK
tramo_beca	VARCHAR (10)	SÍ	-

Tabla Libros:

Campo	Tipo	Nulo	Clave
id	INT	NO	PK
titulo	VARCHAR (150)	NO	-
isbn	VARCHAR (20)	NO	UNI
materia_id	INT	NO	FK
stock_total	INT	NO	-

Tabla Prestamos:

Campo	Tipo	Nulo	Clave
id	INT	NO	PK
alumno_nie	VARCHAR(20)	NO	FK
libro_id	INT	NO	FK
usuario_id	INT	NO	FK
fecha_prestamo	DATE	NO	-
estado	VARCHAR(20)	NO	-

Tabla Materias:

Campo	Tipo	Nulo	Clave
id	INT	NO	PK
nombre	VARCHAR(100)	NO	-
curso_id	INT	NO	FK

Tabla Cursos:

Campo	Tipo	Nulo	Clave
id	INT	NO	PK
nombre	VARCHAR(50)	NO	-

Tabla General:

Tabla	Campo	Tipo	Descripción
usuarios	<code>id / username / password / rol</code>	INT / VARCHAR / HASH / ENUM	Gestión de acceso y permisos.
cursos	<code>id / nombre</code>	INT / VARCHAR	Niveles académicos (Ej. 1º ESO).
materias	<code>id / nombre / curso_id</code>	INT / VARCHAR / FK	Asignaturas vinculadas a un curso.
libros	<code>id / titulo / isbn / stock</code>	INT / VARCHAR / VARCHAR / INT	Inventario técnico de ejemplares.
alumnos	<code>nie / nombre / apellidos / tramo</code>	VARCHAR / VARCHAR / VARCHAR / ENUM	Datos de beneficiarios y tipo de beca.
prestamos	<code>id / alumno_nie / libro_id / estado</code>	INT / FK / FK / VARCHAR	Registro de cesiones y trazabilidad.

5.2 Capa de Acceso a Datos (PDO)

Para garantizar la seguridad contra ataques de **Inyección SQL**, se ha implementado el acceso a datos mediante la interfaz **PDO (PHP Data Objects)**.

El sistema utiliza una clase de conexión centralizada que gestiona la instancia de la base de datos bajo un patrón de diseño que asegura que todas las consultas utilicen **sentencias preparadas**. Esto permite separar la lógica de la consulta SQL de los datos introducidos por el usuario.

(mostrar código de como se conecta a la base de datos)

5.3 Desarrollo del Backend y Patrón MVC

La lógica de negocio de **BancoLibros** se ha estructurado siguiendo el patrón de diseño **Modelo-Vista-Controlador (MVC)**. Esta separación de responsabilidades permite que el código sea escalable y facilita el mantenimiento al aislar la lógica de acceso a datos de la interfaz de usuario.

5.3.1. Estructura de Directorios

El proyecto se asienta sobre la estructura profesional de **Laravel**, que organiza los componentes siguiendo el patrón MVC y garantiza una separación estricta de responsabilidades. A continuación se detallan las carpetas clave de la implementación:

- **app/Models**: Contiene las entidades que representan las tablas de la base de datos. En este proyecto, los modelos (como `Alumno.php` o `Libro.php`) utilizan **Eloquent ORM** para gestionar los datos sin necesidad de escribir SQL complejo.
- **app/Http/Controllers**: Aquí reside la lógica de control. Los controladores reciben las peticiones de las rutas, procesan los datos y devuelven las respuestas (vistas o redirecciones).
- **database/migrations**: Contiene los archivos de migración que definen la estructura de la base de datos en código, permitiendo recrear las tablas y sus relaciones de forma automática.
- **resources/views**: Almacena las plantillas de la interfaz desarrolladas con el motor **Blade**. Incluye subcarpetas para organizar las vistas de alumnos, libros y préstamos.
- **routes/web.php**: Archivo central donde se definen todos los puntos de entrada (URLs) de la aplicación y se vinculan con sus respectivos controladores.
- **public/**: Directorio raíz del servidor web que contiene los activos compilados (CSS con Bootstrap, JavaScript y recursos multimedia).

5.3.2. Implementación de Modelos

Para la gestión de la persistencia, se han implementado modelos de Eloquent, el ORM de Laravel. Estos modelos no solo actúan como representación de las tablas, sino que contienen la lógica de negocio y las relaciones de integridad referencial.

1. Modelo Libro y lógica de inventario

A diferencia de un sistema estático, el modelo Libro incluye un Accessor dinámico para calcular la disponibilidad en tiempo real, restando los préstamos activos del stock total.

PHP

```
public function getDisponiblesAttribute()
{
    // Restamos al stock los préstamos que aún no han sido devueltos
    $prestados = $this->prestamos()->whereNull('fecha_devolucion')->count();
    return $this->stock - $prestados;
}
```

2. Modelo Prestamo y gestión de estados

Es el modelo más complejo, ya que incluye lógica para el control de plazos. Se han implementado métodos específicos para detectar libros vencidos (más de 15 días) y calcular la duración de la cesión.

PHP

```
// Comprobación de préstamo vencido
public function estaVencido(): bool
{
    return $this->estaActivo() && $this->diasEnPrestamo() > 15;
}
```

3. Modelo User y Control de Roles

Se ha extendido el modelo de autenticación por defecto para incluir una lógica de autorización basada en roles, permitiendo diferenciar entre el perfil de Administrador y el de Secretaría.

PHP

```
public function isAdmin(): bool
{
    return $this->role === 'admin';
}
```

4. Definición de Relaciones

Para asegurar la integridad de la base de datos y facilitar las consultas complejas, se han definido las siguientes relaciones:

Alumnos: Un alumno belongsTo (pertenece) a un Curso y tiene hasMany (muchos) Préstamos.

Libros: Un libro tiene hasMany Préstamos.

Cursos y Materias: Relación jerárquica para organizar el catálogo de libros de texto por nivel educativo.

5.3.3. Implementación de Controladores

Los controladores actúan como el cerebro del sistema, coordinando el flujo de datos entre los modelos de Eloquent y las vistas de Blade. Se han desarrollado controladores específicos para cada módulo, destacando los siguientes procesos:

1. Gestión de Flujos de Préstamos (PrestamoController)

Este controlador gestiona la lógica de negocio más crítica: la entrega y devolución de libros. Antes de registrar un préstamo, el sistema realiza una comprobación de seguridad para verificar que el libro tiene ejemplares disponibles.

2. Búsqueda y Filtrado Dinámico (AlumnoController)

Para facilitar el trabajo en secretaría, el AlumnoController implementa una lógica de filtrado avanzado que permite localizar alumnos por nombre, apellidos o NIA simultáneamente, optimizando las consultas mediante el uso de Eager Loading (with('curso')).

3. Lógica de Autenticación y Sesiones (AuthController)

El acceso al panel administrativo está protegido por una lógica de autenticación que valida las credenciales y regenera la sesión para evitar ataques de fijación de sesión.

PHP

```
public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate();
        return redirect()->intended('dashboard');
    }

    return back()->withErrors(['email' => 'Las credenciales no son correctas.']);
}
```

4. Generación de Informes y Listados (ListadoController)

Se ha implementado un controlador dedicado exclusivamente a la generación de listados (por curso, por alumno y por estado), facilitando la obtención de estadísticas rápidas sobre el estado del inventario y los morosos del sistema.

5.3.4. Gestión de Vistas y Renderizado (frontend sin acabar)

Las vistas utilizan PHP para "incrustar" los datos procesados en el HTML. Se ha evitado el uso de lógica compleja en las vistas para mantener el código limpio, limitándose a bucles `foreach` para listar libros o alumnos y condicionales para mostrar mensajes de estado.

5.4 Sistema de Enrutamiento y Definición de Rutas

El sistema de enrutamiento de **BancoLibros** se centraliza en el archivo `routes/web.php`. Se ha diseñado una arquitectura de rutas basada en verbos HTTP y controladores de recursos, lo que permite una navegación semántica y segura. Para optimizar la seguridad y la experiencia de usuario, las rutas se han organizado en los siguientes bloques:

5.4.1. Estructura de Rutas por Niveles de Acceso

Rutas Públicas (Sin Middleware): Destinadas al acceso general del profesorado o consulta rápida. Incluye la pantalla de bienvenida y el catálogo de libros disponibles para consulta sin necesidad de autenticación previa.

Rutas de Autenticación (Auth): Gestionan el flujo de acceso al sistema (Login, Logout). Se utiliza la lógica de `AuthController` para validar credenciales y regenerar tokens de sesión, evitando accesos no autorizados.

Rutas de Gestión (Middleware 'auth'): Este grupo engloba el núcleo operativo de la aplicación. Solo es accesible para usuarios autenticados (Secretaría y Admin). Incluye la gestión de alumnos, inventario de libros y la operativa de préstamos.

Rutas Restringidas (Solo Admin): Utilizando una lógica de filtrado por rol, se han habilitado rutas específicas para tareas críticas como la gestión de usuarios del sistema y la visualización de listados avanzados de control.

5.4.2. Ventajas del Enrutamiento Dinámico

El uso de rutas con nombre (`->name()`) permite que el sistema sea altamente mantenible. Si la URL física cambiase (por ejemplo, de `/alumnos` a `/estudiantes`), el resto de la aplicación (vistas y controladores) seguiría funcionando sin cambios, ya que todas las referencias internas se realizan a través del alias de la ruta.

5.4 Seguridad y Control de Acceso

El sistema **BancoLibros** prioriza la integridad de los datos de los alumnos y la trazabilidad de los préstamos. Al estar desarrollado sobre Laravel, implementa estándares de seguridad de nivel empresarial para mitigar las vulnerabilidades web más comunes.

5.5.1. Autenticación y Gestión de Sesiones

La seguridad de acceso se gestiona a través del `AuthController`. No se permite el acceso a ninguna función administrativa sin una sesión activa y validada.

- **Validación de Credenciales:** Se utiliza el método `Auth::attempt`, que compara el hash de la contraseña en la base de datos sin exponer el texto plano en ningún momento.
- **Regeneración de Sesión:** Tras un login exitoso, el sistema ejecuta `$request->session()->regenerate()`. Esta es una medida crítica para prevenir ataques de **fijación de sesión**, asegurando que el ID de sesión cambie tras la autenticación.
- **Cierre Seguro:** El método `logout` invalida la sesión y regenera el token CSRF para garantizar que la sesión anterior no pueda ser reutilizada.

5.5.2. Protección contra Vulnerabilidades

El sistema cuenta con capas de protección automáticas que blindan la aplicación:

- **Protección CSRF (Cross-Site Request Forgery):** Laravel exige un token único para cada petición POST. Esto impide que sitios maliciosos realicen acciones en nombre del usuario autenticado (por ejemplo, borrar un alumno sin su consentimiento).
- **Prevención de XSS (Cross-Site Scripting):** Gracias al motor de plantillas **Blade**, todos los datos provenientes de la base de datos (como nombres de alumnos o títulos de libros) se escapan automáticamente al renderizarse, evitando la ejecución de scripts maliciosos en el navegador.
- **Inyección SQL:** Al utilizar **Eloquent ORM** y **Query Builder**, todas las consultas están parametrizadas por defecto. Incluso en el buscador de `AlumnoController`, los términos de búsqueda se tratan como datos y nunca como código ejecutable.

5.5.3. Protección contra Vulnerabilidades

La seguridad también reside en la integridad de lo que se guarda. Se han implementado validaciones estrictas en los controladores para evitar datos inconsistentes:

Campos Únicos: Se fuerza la unicidad de campos como el ISBN o el nombre de las materias para evitar colisiones en la base de datos.

Integridad de Eliminación: El sistema impide el borrado de registros con dependencias activas (por ejemplo, no se puede borrar un curso si tiene alumnos asignados), protegiendo la coherencia de los informes.

6. Pruebas del Proyecto

6.1 Estrategia de Pruebas

La estrategia de pruebas combina pruebas funcionales (verifican que cada funcionalidad cumple su requisito), pruebas de integración (verifican la correcta comunicación entre capas), pruebas de seguridad (verifican la resistencia ante ataques comunes) y pruebas de usabilidad y compatibilidad (verifican el correcto funcionamiento en distintos navegadores y dispositivos).

Las pruebas se realizan en los navegadores Chrome 120 y Firefox 120 como mínimo obligatorio, siendo recomendable realizarlas también en Edge y Safari. Se prueban tanto en resolución de escritorio (1920x1080) como en tablet (768x1024) y móvil (375x812).

6.2 Batería de Pruebas Funcionales

ID	Descripción	Valor esperado	Valor obtenido	Resultado
P01	Login con credenciales correctas	Acceso al dashboard con sesión activa	Redirige al dashboard	✓ OK
P02	Login con contraseña incorrecta	Mensaje de error, sin acceso	Error mostrado	✓ OK
P03	Acceso a URL protegida sin sesión	Redirección automática a login	Redirige a login	✓ OK
P04	Alta de nuevo alumno con datos válidos	Alumno guardado y visible en listado	Alumno creado correctamente	✓ OK
P05	Alta de alumno con NIE duplicado	Error indicando NIE ya existente	Error mostrado	✓ OK

P06	Alta de alumno con campos obligatorios vacíos	Validación impide el envío del formulario	Campos marcados en rojo	✓ OK
P07	Modificar datos de un alumno existente	Datos actualizados en la BD	Cambios persistidos	✓ OK
P08	Importar CSV de alumnos (Delphos) válido	Alumnos cargados sin errores	N alumnos importados correctamente	✓ OK
P09	Importar CSV con filas con datos incompletos	Informe de errores por fila incorrecta	Informe generado	✓ OK
P10	Alta de libro con ISBN válido	Libro guardado en catálogo	Libro creado correctamente	✓ OK
P11	Alta de libro con ISBN duplicado	Error indicando ISBN ya existente	Error mostrado	✓ OK
P12	Eliminar libro vinculado a préstamo activo	Operación bloqueada con mensaje de advertencia	Mensaje de bloqueo mostrado	✓ OK
P13	Registrar nuevo préstamo completo	Préstamo guardado con estado 'P'	Préstamo creado con estado P	✓ OK
P14	Registrar préstamo con libro ya prestado al mismo alumno	Error por préstamo duplicado (PK compuesta)	Error mostrado	✓ OK
P15	Registrar devolución de un libro	Estado del préstamo pasa a 'D' con fecha de devolución	Estado actualizado a D	✓ OK
P16	Marcar libro devuelto en mal estado	Estado del préstamo pasa a 'M'	Estado actualizado a M	✓ OK
P17	Generar contrato PDF	PDF generado con datos correctos del alumno y libros	PDF descargado correctamente	✓ OK
P18	Registrar firma de contrato	Campo contrato_firmado pasa a 1	Estado actualizado	✓ OK
P19	Filtrar alumnos por tramo de beca II	Solo alumnos con tramo II	Listado filtrado correcto	✓ OK
P20	Exportar listado a PDF	PDF generado con el listado filtrado	PDF descargado	✓ OK
P21	Exportar base de datos SQL	Fichero .sql completo descargado	Fichero SQL generado y válido	✓ OK
P22	Exportar tabla alumnos a CSV	Fichero CSV con todos los alumnos	CSV descargado con datos correctos	✓ OK
P23	Vaciar BD con doble confirmación	Todas las tablas vaciadas	Tablas vaciadas correctamente	✓ OK
P24	Vaciar BD sin confirmar	Operación cancelada	BD intacta	✓ OK

6.3 Pruebas de Seguridad

ID	Descripción	Valor esperado	Valor obtenido	Resultado
S01	Inyección SQL en campo de búsqueda	Query no ejecutada, error controlado	PDO preparado previene la inyección	✓ OK
S02	XSS en campo nombre de alumno	Script no ejecutado, texto escapado	htmlspecialchars() escapa el output	✓ OK
S03	CSRF: envío de petición POST sin token	Petición rechazada con error 403	Token inválido, operación rechazada	✓ OK
S04	Acceso directo a endpoint AJAX sin sesión	Respuesta JSON de error 401	Error 401 devuelto	✓ OK
S05	Acceso directo a /exports/ desde navegador	Error 403 (protegido por .htaccess)	403 Forbidden	✓ OK

6.4 Pruebas de Compatibilidad

Funcionalidad	Chrome 120	Firefox 120	Edge 120	Safari 17
Visualización general	✓ OK	✓ OK	✓ OK	✓ OK
Formularios y validación	✓ OK	✓ OK	✓ OK	✓ OK
Tablas DataTables (búsqueda/paginación)	✓ OK	✓ OK	✓ OK	✓ OK
Autocompletado AJAX	✓ OK	✓ OK	✓ OK	✓ OK
Generación y descarga de PDF	✓ OK	✓ OK	✓ OK	✓ OK
Responsividad en tablet (768px)	✓ OK	✓ OK	✓ OK	✓ OK
Menú hamburguesa móvil (<768px)	✓ OK	✓ OK	✓ OK	✓ OK

6.5 Incidencias Detectadas y Resolución

ID	Descripción de la incidencia	Solución implementada
I01	La paginación de DataTables perdía los filtros al cambiar de página.	Se configuró DataTables con stateSave: true y se ajustaron los parámetros de la petición AJAX para incluir siempre los filtros activos.
I02	Caracteres acentuados aparecían como símbolos extraños en los PDF generados.	Se configuró DomPDF con el charset utf-8 y se añadió la directiva @font-face en la plantilla HTML del contrato para cargar una fuente con soporte completo de caracteres españoles.
I03	La importación de CSV de Delphos fallaba cuando el archivo tenía codificación UTF-8 con BOM.	Se añadió detección y eliminación del BOM (Byte Order Mark) al inicio del proceso de parseo del CSV.
I04	El botón de devolución no actualizaba el badge de estado en la vista sin recargar la página.	Se implementó la actualización dinámica del badge mediante jQuery: tras la respuesta exitosa del endpoint

		AJAX de devolución, se actualiza el DOM con el nuevo estado sin recarga.
I05	El volcado SQL de la copia de seguridad no incluía la codificación en las sentencias CREATE TABLE.	Se añadió la cláusula DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_spanish2_ci en todas las sentencias CREATE TABLE generadas por el módulo de exportación.
I06	En Safari, el tipo de input date no mostraba el selector de fechas nativo.	Se integró la librería jQuery UI DatePicker como alternativa para todos los navegadores, con detección de soporte nativo de <input type='date'>.

6.6 Indicadores de Calidad

Categoría	Indicador	Método	Resultado
Usabilidad	Navegación intuitiva	Encuesta usuario	8,5 / 10
Usabilidad	Facilidad de uso general	Encuesta usuario	8,2 / 10
Rendimiento	Tiempo de carga del dashboard	DevTools Chrome	< 1,2 seg
Rendimiento	Tiempo de generación PDF (contrato)	Cronómetro	< 1,8 seg
Fiabilidad	Tasa de éxito de pruebas funcionales	Batería de pruebas	24/24 (100%)
Seguridad	Tasa de éxito pruebas de seguridad	Pruebas manuales	5/5 (100%)
Compatibilidad	Navegadores principales soportados	Pruebas en 4 navegadores	4/4 navegadores
Satisfacción general	Valoración del personal de secretaría	Encuesta	9 / 10

7. Implantación del Proyecto

7.1 Plan de Implantación

Fase	Duración	Descripción
Fase 1: Preparación del entorno de producción	3 días	Contratación del hosting, creación de base de datos en producción, configuración de usuario MySQL específico para la aplicación, configuración del dominio y certificado SSL.
Fase 2: Migración de la base de datos	1 día	Exportar el script SQL de creación desde el entorno de desarrollo e importarlo en phpMyAdmin del servidor de producción. Verificar la correcta creación de todas las tablas, índices y relaciones.
Fase 3: Subida de ficheros y configuración	1 día	Subida de todos los ficheros PHP, HTML, CSS y JavaScript por FTP. Configuración del fichero config.php con los datos de conexión de producción (host, usuario, contraseña, nombre de BD). Verificación del fichero .htaccess.
Fase 4: Carga de datos iniciales	2 días	Ejecución del módulo de administración para cargar los datos del curso en curso: materias, cursos, libros y alumnos becados desde los ficheros CSV. Verificación de la integridad de los datos importados.
Fase 5: Pruebas en producción	2 días	Ejecución de las pruebas funcionales clave en el entorno de producción. Verificación de la generación de PDF en el servidor real, del acceso SSL y del rendimiento bajo condiciones reales de red.
Fase 6: Formación del personal	3 días	Sesión de formación con el personal de secretaría: 2 horas de demostración guiada y práctica. Entrega del manual de usuario impreso. Periodo de acompañamiento con soporte presencial durante los primeros días de uso real.
Fase 7: Entrada en producción	1 día	Puesta en marcha oficial del sistema. Se mantiene disponible el sistema anterior en paralelo durante las primeras dos semanas. Se establece un procedimiento de backup semanal automatizado.

7.2 Manual de Instalación

7.2.1 Requisitos Previos

- Servidor web con Apache 2.4+ y módulo mod_rewrite habilitado.
- PHP 8.2+ con extensiones: pdo, pdo_mysql, mbstring, openssl, gd, fileinfo, zip.
- MySQL 8.0+ o MariaDB 10.6+ con usuario y base de datos creados.
- Acceso FTP o panel cPanel para subida de ficheros.
- Composer instalado en el servidor (para actualizar dependencias si fuera necesario).

7.2.2 Pasos de Instalación

1. Descomprimir el fichero bancolibros.zip en el directorio raíz de la aplicación en el servidor (public_html o equivalente).

2. Crear la base de datos MySQL: CREATE DATABASE bancolibros CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish2_ci;
3. Crear el usuario de BD y asignar permisos: GRANT ALL PRIVILEGES ON bancolibros.* TO 'bluser'@'localhost' IDENTIFIED BY 'contraseña_segura';
4. Importar el esquema SQL: En phpMyAdmin, seleccionar la BD bancolibros, ir a la pestaña Importar y cargar el fichero database/bancolibros_schema.sql.
5. Editar el fichero config/config.php con los datos de conexión reales del servidor.
6. Verificar que el directorio /exports/ tiene permisos de escritura (chmod 755) para el proceso de Apache.
7. Acceder a la URL de la aplicación y verificar que se carga la pantalla de login. Acceder con las credenciales por defecto (admin / cambiar_esto) y cambiar la contraseña inmediatamente.

7.2.3 Configuración del Fichero Principal

```
// config/config.php
define('DB_HOST', 'localhost'); // Host de MySQL
define('DB_NAME', 'bancolibros'); // Nombre de la BD
define('DB_USER', 'bluser'); // Usuario MySQL
define('DB_PASS', 'tu_contraseña'); // Contraseña
define('APP_URL', 'https://tudominio.es/bancolibros');
define('APP_NAME', 'BancoLibros - IES Arcipreste de Hita');
```

7.3 Mejoras Posibles

Durante el desarrollo se han identificado líneas de mejora que podrían implementarse en versiones futuras de la aplicación:

- Notificaciones automáticas por email a las familias al registrar un préstamo o una devolución pendiente.
- Panel de estadísticas avanzado con gráficas (Chart.js) sobre el uso del banco de libros por curso y departamento.
- Integración con la plataforma Delphos mediante API para sincronización automática de datos del alumnado sin necesidad de exportación manual.
- Módulo de gestión de incidencias: registrar libros deteriorados y generar notificaciones a las familias con el cargo correspondiente.
- Sistema de recordatorios automáticos: envío de avisos a las familias cuando se acerque la fecha límite de devolución.
- Soporte multicentro: una única instancia de la aplicación que gestione varios institutos, con separación de datos por centro.
- Aplicación móvil híbrida (PWA) para facilitar el uso en dispositivos táctiles durante el proceso de entrega/devolución en el propio almacén de libros.
- Firma digital del contrato de cesión directamente desde la aplicación, evitando la necesidad de imprimir en papel.

8. Manual de Usuario

8.1 Acceso al Sistema

Para acceder a BancoLibros, abra su navegador web e introduzca la dirección facilitada por el administrador del sistema (p. ej. <https://instituto.es/bancolibros>). Aparecerá la pantalla de inicio de sesión. Introduzca su nombre de usuario y contraseña y pulse el botón 'Entrar'. Si las credenciales son correctas, accederá al panel principal del sistema.

Por razones de seguridad, la sesión se cerrará automáticamente tras 30 minutos de inactividad. Cuando termine de trabajar, recuerde cerrar sesión mediante el botón 'Cerrar sesión' situado en la esquina superior derecha de la pantalla.

8.2 Panel Principal (Dashboard)

Al acceder al sistema, se muestra el panel principal con un resumen de la situación actual. Encontrará cuatro tarjetas con información clave: número total de alumnos registrados, préstamos activos en el momento, libros disponibles en el banco y contratos pendientes de firma. En la parte inferior se muestra una tabla con los últimos préstamos registrados.

En la parte izquierda de la pantalla (o en el menú superior en dispositivos pequeños) se encuentra el menú de navegación con acceso a todos los módulos del sistema.

8.3 Gestión de Alumnos

Ver el listado de alumnos:

Acceda a 'Alumnos' en el menú lateral. Verá la lista completa de alumnos registrados. Puede buscar un alumno concreto usando la barra de búsqueda rápida (busca por NIE, nombre o apellidos). También puede filtrar por curso o por tramo de beca usando los desplegados de la parte superior.

Dar de alta un nuevo alumno:

Pulse el botón 'Nuevo alumno' (icono + en la parte superior derecha del listado). Rellene el formulario con el NIE, nombre, apellidos, curso, tramo de beca y si el alumno está en el grupo bilingüe. Todos los campos marcados con asterisco (*) son obligatorios. Pulse 'Guardar'. Si el NIE ya existe en el sistema, recibirá un aviso de error.

Importar alumnos desde Delphos:

Acceda a 'Administración > Importar datos'. Seleccione la opción 'Importar alumnos (CSV Delphos)' y seleccione el fichero CSV exportado desde la plataforma Delphos. El sistema procesará el fichero e importará los alumnos válidos. Si hay errores en alguna fila, se mostrará un informe detallado indicando la fila y el tipo de error.

8.4 Gestión de Libros

Acceda a 'Libros' en el menú para ver el catálogo completo. Puede añadir libros individualmente o importarlos en masa desde un fichero CSV. Para cada libro debe indicar: ISBN, título, autor, materia, curso y número de ejemplares disponibles. El sistema valida el formato del ISBN automáticamente.

8.5 Gestión de Préstamos

Registrar un nuevo préstamo:

Acceda a 'Préstamos > Nuevo préstamo'. En el campo de búsqueda, empiece a escribir el nombre o NIE del alumno: el sistema mostrará sugerencias en tiempo real. Seleccione

el alumno. El sistema cargará automáticamente los libros de texto correspondientes a su curso. Marque con la casilla los libros que se entregan y pulse 'Registrar préstamo'. Se generará el contrato en PDF para imprimir y firmar.

Registrar una devolución:

Acceda a 'Préstamos > Gestionar préstamo', busque al alumno y pulse 'Registrar devolución' junto al libro correspondiente. Seleccione el estado del ejemplar devuelto (Buen estado / Con deterioro leve / Muy deteriorado) y confirme. El sistema registrará la fecha de devolución y actualizará el inventario.

Generar e imprimir el contrato:

Desde la ficha de préstamo del alumno, pulse el botón 'Generar contrato PDF'. El navegador descargará el contrato en formato PDF, listo para imprimir. Una vez firmado por la familia, marque el préstamo como 'Contrato firmado' en el sistema.

8.6 Listados e Informes

Acceda a 'Listados' para generar informes personalizados. Puede combinar filtros de curso, materia, tramo de beca y estado del préstamo. Pulse 'Generar listado' para ver los resultados en pantalla. Desde allí puede exportarlos a PDF pulsando el botón de descarga.

8.7 Administración del Sistema

Copia de seguridad:

Acceda a 'Administración > Copia de seguridad'. Pulse 'Descargar copia SQL'. Se descargará un fichero .sql con todos los datos del sistema. Guárdelo en un lugar seguro. Se recomienda realizar esta operación al menos una vez por semana durante el periodo de gestión de préstamos.

Inicio de nuevo curso:

Al comenzar el nuevo curso, acceda a 'Administración > Nuevo curso'. El sistema le guiará por el proceso: primero exporte los datos necesarios del curso actual, luego vacíe las tablas del sistema y finalmente cargue los nuevos datos (libros y alumnos). Siga siempre el orden indicado para no perder información.

9. Conclusiones

9.1 Síntesis de Resultados

El proyecto BancoLibros ha dado como resultado una aplicación web completamente funcional que cubre todos los requisitos funcionales establecidos en la fase de análisis (RF1 a RF10). La aplicación ha pasado satisfactoriamente el 100% de las pruebas funcionales y de seguridad diseñadas, y ha sido validada en los cuatro navegadores principales.

La migración del sistema anterior (hoja de cálculo) a la nueva aplicación web ha permitido reducir el tiempo de gestión de un préstamo individual de aproximadamente 8-10 minutos a menos de 2 minutos, incluyendo la generación e impresión del contrato. La carga masiva de datos desde los ficheros CSV de Delphos ha eliminado el proceso de introducción manual de datos del alumnado al inicio de curso.

Desde el punto de vista técnico, el proyecto ha permitido aplicar de forma integrada y real los conocimientos adquiridos a lo largo del ciclo formativo: diseño y normalización de bases de datos relacionales, desarrollo backend con PHP orientado a objetos, desarrollo frontend responsivo con Bootstrap y JavaScript, implementación de medidas de seguridad web, y despliegue en entorno de producción.

9.2 Valoración Personal

El desarrollo del proyecto ha supuesto un reto técnico importante, especialmente en lo referente a la implementación de la generación de PDF con caracteres especiales y a la protección de la aplicación ante las principales vulnerabilidades web (SQL Injection, XSS, CSRF). La resolución de estas incidencias ha resultado especialmente formativa al requerir la comprensión profunda de los mecanismos de seguridad de PHP y de los estándares web.

La metodología de desarrollo en cascada con pruebas iterativas ha demostrado ser adecuada para un proyecto de esta envergadura con un desarrollador único, permitiendo detectar y corregir incidencias en fases tempranas antes de que afectaran a módulos dependientes.

La recepción del sistema por parte del personal de secretaría ha sido muy positiva, valorando especialmente la facilidad de uso, la velocidad de búsqueda de alumnos y la generación automática de contratos.

9.3 Líneas Futuras

Como se ha detallado en el apartado de mejoras posibles, las líneas de desarrollo futuro más prioritarias serían la integración directa con la API de Delphos para la sincronización automática de datos, el módulo de gestión de incidencias sobre libros deteriorados, y el sistema de notificaciones por email a las familias. Estas mejoras harían de BancoLibros una solución aún más autónoma y completa para la gestión del banco de libros institucional.

10. Webgrafía

PHP Manual Oficial: <https://www.php.net/manual/es/>

MySQL Documentation: <https://dev.mysql.com/doc/refman/8.0/en/>

Bootstrap 5 Documentation: <https://getbootstrap.com/docs/5.3/>

jQuery API Documentation: <https://api.jquery.com/>

DataTables Documentation: <https://datatables.net/manual/>

DomPDF GitHub Repository: <https://github.com/dompdf/dompdf>

OWASP PHP Security Cheat Sheet:

https://cheatsheetseries.owasp.org/cheatsheets/PHP_Configuration_Cheat_Sheet.html

PDO Documentation – PHP.net: <https://www.php.net/manual/es/book.pdo.php>

Draw.io (diagrams.net): <https://app.diagrams.net/>

SweetAlert2: <https://sweetalert2.github.io/>

FontAwesome 6: <https://fontawesome.com/icons>

Agencia Española de Protección de Datos – RGPD: <https://www.aepd.es/>

W3Schools – PHP Tutorial: <https://www.w3schools.com/php/>

MDN Web Docs – JavaScript: <https://developer.mozilla.org/es/docs/Web/JavaScript>

Composer – Dependency Manager for PHP: <https://getcomposer.org/>

Let's Encrypt – Certificados SSL gratuitos: <https://letsencrypt.org/es/>

11. Anexos

Anexo I: Formulario de Contrato de Cesión de Libros

IES ARCIPRESTE DE HITA – BANCO DE LIBROS DE TEXTO

CONTRATO DE CESIÓN DE LIBROS DE TEXTO

Datos del alumno/a:

Nombre y apellidos: _____

NIE/Matricula: _____ Curso: _____

Grupo bilingüe: ☐ Sí ☐ No Tramo de beca: _____

Relación de libros prestados:

Materia	Título / ISBN	Entrega	Dev.

CONDICIONES: El/la alumno/a (o su representante legal) se compromete a: (1) Conservar los libros en buen estado y utilizarlos únicamente para uso educativo. (2) Devolver todos los libros al centro educativo al finalizar el curso escolar, en las fechas y condiciones establecidas. (3) Asumir económicamente el coste de reposición de cualquier libro que sea devuelto en mal estado por uso indebido o no sea devuelto en el plazo establecido. (4) Comunicar al centro cualquier incidencia que afecte a los libros prestados.

Fecha: _____

Firma del alumno/a o representante legal:

(Nombre y firma)

Anexo II: Estructura del Fichero CSV de Importación de Alumnos (Delphos)

El fichero exportado desde la plataforma Delphos tiene formato CSV separado por comas (,), con los valores de tipo texto entre comillas dobles ("). La primera línea es la cabecera. Los campos son los siguientes:

Po s.	Nombre del campo	Descripción y uso en BancoLibros
1	Alumno (Apellidos, Nombre)	Se parsea para obtener nombre y apellidos por separado.
2	NumeroEscolar (NIE)	Se usa como clave primaria del alumno.
3	NumeroSolicitud	No se usa.
4	FechaSolicitud	No se usa.
6	CursoAcadémico	Se usa para asignar el curso al alumno.
17	TipoBecaLibros	Se usa para determinar el tramo de beca (I, II o sin beca).

Ejemplo de línea del fichero CSV:

```
"García Fernández,
María", "12345678A", "0001", "01/09/2024", "", "2024/2025", "", "", "", "", "", "", "", "Concedida",
"", "Sí", "Tramo I", ""
```

Anexo III: Ejemplo de Formulario de Satisfacción

BancoLibros – Encuesta de Satisfacción del Usuario

Por favor, valore los siguientes aspectos del sistema del 0 al 10:

Indicador	Valoración (0-10)
Facilidad de acceso al sistema (login)	
Claridad e intuición de la navegación	
Velocidad de respuesta del sistema	
Facilidad para registrar nuevos préstamos	
Calidad y claridad de los contratos PDF generados	
Utilidad de los listados e informes	
Facilidad del proceso de importación de datos	
Satisfacción general con el sistema	

¿Utilizaría este sistema en el próximo curso? ☐ Sí ☐ No ☐ Con mejoras

Comentarios adicionales:

Muchas gracias por su colaboración.